Unveiling the Nexus: Harnessing IoT for Internet Censorship Evasion Through Fragmentation

Anonymous Author(s)

Abstract

In recent years, numerous research breakthroughs have emerged in the realm of anti-censorship, including automated methods to probe and generate censorship circumvention policies, network packet manipulation, and cryptography. Among the practical implementations of circumvention systems, centralized design prevails as the most popular approach, which mainly utilizes a single routing entity (servers, data centers, etc.) with predetermined IP addresses and cryptographic protocols. However, despite popularity, some but not all centralized circumvention models fall short of ad hoc censorship systems, meaning that such systems are sometimes designed with a specific region (China, Turkmenistan, Russia, etc.) in mind. Such adversaries sometimes enforce Internet blockades through mass IP range blocking and explicitly designed detection heuristics targeting protocols known to be mainly used for censorship circumvention. In response, we introduce a new approach with *Nexus Packet Fragmentation Algorithm*. Alongside, we also present a theoretical circumvention model, *Nexus Network*, based on a low-cost, semi-autonomous IoT-based distributed design without the need for centralized servers for coordination or routing. We present our algorithm as the core of our future design to offload routing and obfuscation tasks to dispersed IoT devices across networks, which possess computational power and vast IP spaces in unison. At the current stage, we show that our algorithm bypasses censorship systems by sending packets in dissected fragments with lightweight algorithms based on bit-manipulations, enforcing entropy patterns, and smoothing packet lengths. We tested the rudimentary fragmentation algorithm on our testbed in China and found that the algorithm successfully enabled circumvention through the Great Firewall (GFW) in China and accessed 100% of websites and resources listed by the Citizen Lab while deployed on low-power IoT devices. We demonstrate that the low-cost IoT network approach is valid against the current state-of-the-art of censorship techniques. In the end, the fragmentation algorithm serves as the stepping stone towards implementing the full distributed design of *Nexus*, offering a new path forward to address the inefficiencies associated with both distributed and centralized circumvention approaches.

Keywords

IoT, censorship, distributed network, packet fragmentation

1 Introduction

Internet censorship has been a prevalent and ever-growing issue around the world, now not limited to those countries nominally known for such practices, including China, Iran, and Russia [4, 5, 40], but present even among countries that are perceived to enforce little Internet censorship [32], increasing the urgency of the matter. Across the globe, Russia and China are prominent censorship actors, implementing censorship through a wide range of IP blocking, DNS pollution, or dropping unauthorized connections to prohibited services detected via fingerprinting censorship-specific protocol [17, 36, 40]. In addition, although much research has been published throughout the past few years, it was mostly composed of exposing vulnerabilities within censorship systems—such as the GFW of China—that can be exploited for circumvention [14, 36], but fell short in offering a new direction to exploit these novel findings in practice.

Incidentally, the rapid application of Internet of Things (IoT) devices offers valuable means for scientific testbeds, data collection, and data storage due to their low cost, low power consumption, and billions of IP addresses around the world [2, 9]. This paves a new pathway to a volunteer-based, distributed, and self-driven network for censorship circumvention routing without any centralized or static servers.

Hence, this paper combines the two subjects of censorship circumvention and IoT-based distributed systems to devise a new method, deviating from traditional mimicry-based circumvention techniques[18].

We propose *Nexus*, a new censorship circumvention model based on a theoretical distributed IoT network, and implement the *Nexus Packet Fragmentation Algorithm* as the initial touchstone to validate the foundations. With our design composed of an IoT-based distributed network, *Nexus* could potentially hinder Deep Packet Inspection (DPI), Device Fingerprinting (DFP), and other traffic-correlation analysis sanctioned by nation-states [36] by fragmenting packets into separate packets and sending them across different routes, each containing only parts of the original whole packet. This can be achieved at a fraction of the cost, thanks to the economical nature of IoT devices and therefore vast deployments across IP ranges. This will be explained in Section 3.1.

We hypothesize possible attack models based on a comprehensive review of previous work ranging from ML classifiers deployed to detect high-throughput obfuscated traffic [23, 25] to known techniques (e.g., active probing and deep-learning ML models) utilized by nation-states to fingerprint possible VPN tunnels [33, 41]. With our example algorithm for the worst-case scenario where there is no encryption on the payload at all, it fragments plain-text packets by sensitive keywords at the byte level, pads each packet to flatten the IoT fingerprints—most notably packet length and stream direction [8, 23, 25, 30]. *Nexus* is proposed to circumvent the heuristics implemented by the GFW [36], and sends the fragments through a network of IoT devices in parallel. This can be done with encryption only applied to the custom metadata header but not the payload

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.
Proceedings on Privacy Enhancing Technologies YYYY(X), 1–11
© YYYY Copyright held by the owner/author(s).
https://doi.org/XXXXXXX.XXXXXXX

(unless otherwise specified). Additionally, the users receive the subset nodes from *Nexus Network*'s distribution server over a lossy channel with a fountain-code-esque design based on the design by Cawthon et al [7].

2 Background

Traditionally, centralized VPN services have been the predominant means of Internet circumvention, mainly through popular protocols such as *OpenVPN* and *WireGuard* for ease-of-use, customizability, and robustness [41]. Alternatively, *Tor* presents one of the only general-use decentralized circumvention solutions, which routes clients' Internet traffic through volunteers' PCs through multiple hops to reach the destination [10]. While both methods have experienced varying successes in combating state-sanctioned censorship and making themselves ubiquitous to the general public through open-source, they have critical vulnerabilities in some aspects.

2.1 Centralized Approaches

Although mainstream circumvention protocols are competent in payload obfuscation and simplistic implementations [11, 19], they are prone to fingerprinting, active probing, and detection-by-heuristics [36].

Per previous works by Diwen Xue et al., VPN services utilizing *OpenVPN* are subject to active probing from adversaries by fingerprinting their handshake procedures and responses from specifically constructed payloads [41]. Furthermore, *WireGuard*, a VPN protocol designed for lightweight use, does not focus on obfuscation and is known to be easily detected by DPI due to its consistent fingerprints from the headers [11, 35], allowing the adversary to block the protocol en masse without having to acquire knowledge about the plaintext payload. Given the vulnerability of centralized circumvention designs to targeted blocking and fingerprinting, some researchers have turned to other distributed approaches instead.

2.2 Distributed Approaches

In contrast to the aforementioned centralized circumvention architectures, *Tor*'s design assigns volunteer nodes ("*Tor Bridges*") through a coordinator index server with corresponding keys to ensure secret transmission of the packets via many pluggable transport protocols, including *obfs4* [10]. However, while *Tor* offers a distributed architecture that is challenging for censors to detect or decrypt the packets due to its powerful protocols and distributed design, it poses constraints on performance compatibility with certain volunteer devices and raises certain privacy concerns for the volunteers. To address these issues, the *Tor Project* team also introduced the *Snowflake* architecture, allowing more users to participate (with ephemeral presence in the system) by just using designated browsers embedded with a more lightweight peer-to-peer WebRTC protocol and forwarding traffic to a centralized bridge [6].

However, the persistent existence of a centralized index server in both iterations implies a single point of failure in the entire system, even though it is not used for routing. Although this can be mitigated by distributing the relay list through domain fronting, with the diminishing number of Content Delivery Network (CDN) services that allow domain fronting at all, the concern remains.

In addition, the current design of *Tor Bridge* and *Snowflake* poses potential risks to the client in particular. Although unlikely, should the adversary impersonate both the entry and exit node of a session, they are able to intercept clients' traffic and deanonymize them. Additionally, although Snowflake proved to be user-friendly and a step-up from the original *Tor Bridges* in terms of ease of usage, it had yet to address deployability on low-cost, low-power devices.

Therefore, inspired by the colossal number of IoT devices in the wild, which occupy a much wider IP space [2, 9], the proposed *Nexus* aims to improve the resolutions to these issues through *Nexus Network*, letting a network of semi-autonomous IoT devices collectively act as index servers, whilst transmitting small, fragmented chunks of data packets among each volunteer IoT device with lightweight to no encryption, thus lightening the burden of receiving large chunks of data and decryption done on the volunteer device. As we will explain in Section 5, not only does the lack of encryption not significantly increase the aforementioned risks, but it also, albeit theoretically, provides better performance when fragmented packets are transmitted in parallel.

3 Motivation

Despite being the major core of circumvention throughout the past decades, circumvention-by-mimicry has been argued to be ineffective against massively funded censorship infrastructures [18, 36, 41]. In their previous work, Amir Houmansadr et al. stated that mimicry-based circumvention methods are unsustainable against state-sanctioned censorship heuristics, implying that future circumvention models should diverge from developing circumvention techniques that impersonate traffic as regular services or mere padding schemes [18]. Therefore, we provide *Nexus* as an alternative, focusing on traffic flow manipulation.

In addition, although distributed censorship circumvention models, such as *Tor*, have existed for a while, our model aims to obtain improved cost and scalability with cheaper IoT devices.

3.1 Cost

With designs such as *Tor*'s, it is assumed that the volunteers would deploy the *Tor Bridge* on a Personal Computer (PC) to serve as a "jumping server" to circumvent censorship without going through centralized servers with static IP(s) [10]. While robust and proven to function sufficiently over the decade, its computation-heavy protocol scheme (e.g.,*obfs4*) requires at least somewhat capable devices such as a laptop to run on, which can cost from $100 up to $2,000 in USD [1]. In contrast, a commonly used System-on-Chip (SoC)—a type of bare-bone IoT device—with Wi-Fi capability, namely the ESP32 SoC and Raspberry Pi, would only cost single digits in USD [29]. Although less capable of handling large traffic clusters and computationally heavy encryption functions due to bandwidth and CPU limitations, we aim to reach equal routing performance with a fraction of the cost through *Nexus*.

3.2 Scalability

With the drastic decrease in monetary cost per unit, full utilization of IPv6 space becomes much more attainable. With tens of billions of low-power IoT already in the wild and occupying IPv6 addresses [2], *Nexus* employs the new-found vast network of nodes

to dampen the most commonly used general-use network traffic fingerprinting features—particularly traffic direction [8, 23, 25, 30]—by dissecting the network packets into fragments and simultaneously routing them to distinct volunteer IoT nodes within the distributed network. Such scalability would hinder censors' ability to identify and drop prohibited actions non-trivially without needing fully encrypted channels, and also be compatible with encryption when the delivered content itself requires high confidentiality.

In addition, as discussed in Section 6.1, in comparison with *Tor*, this paper proposes that *Nexus Network* also distributes a partial list of "neighbor" nodes among volunteer nodes themselves. Not only would this method increase the scalability as it shifts the burden away from distribution servers, but it also dampens the possible damage by impersonating nodes since they would not acquire the entire topology of the *Nexus* network.

4 Adversary Model

The adversary we consider in this work has comprehensive control over their regional deployment, censoring through ISP-level on-path DPI (looking for plain keywords or fingerprints) and DFP, IP range blocking, etc [3, 12, 17, 34, 39]. Specifically, we presume that the GFW is implemented on critical Internet infrastructures, such as the Border Gateways of Autonomous Systems that connect the Internet in China to the outside world [28]. The most common symptoms that indicate an occurrence of blocking would be TCP RST attacks and consistent packet drops [34]; we also expect that an adversary may be able to trace the real identity through mere source IP addresses.

In contrast, we also assume that some parts of GFW are regionally deployed [37], and coordination among these regions is relatively infrequent. This implies that it might be implausible for the system to identify that the corresponding fragmented packets are incidentally routed through Autonomous Systems (ASes) in different regions. Because of this, we also assume that these regional adversaries within China do not necessarily have identical or similar Standard Operating Procedures (SOPs). This loophole exposes a potential vulnerability in on-path network surveillance, as provincial-level censorship systems might have different levels of risk-aversion and definitions of suspicious network traffic [37].

Moreover, according to many fingerprinting techniques developed by previous work, mere stream direction (upstream or downstream, and ports connected), packet lengths, and timings would be sufficient to identify traffic of interest without the need to decrypt anything [8, 23, 25, 30, 36]. Thus, we assume that the adversary would implement similar techniques to detect "irregular" traffic from or to non-traditional devices that would not receive or initiate connections across the border.

Lastly, with regard to the concerns surrounding Quantum Computing, many projects have also integrated quantum-resistant algorithms into their protocols preemptively [38]. However, due to a lack of evidence and reports for any deployment of the aforementioned tools or usage of Quantum Computers for censorship, such scenarios are not included in our adversary model.

Therefore, weighing the prominence and novelty factors of the censorship methods discovered, for this paper, we primarily focus on the adversary model in which the censor implements the threat model based mainly on those described in the papers by Wu et al [36, 37].

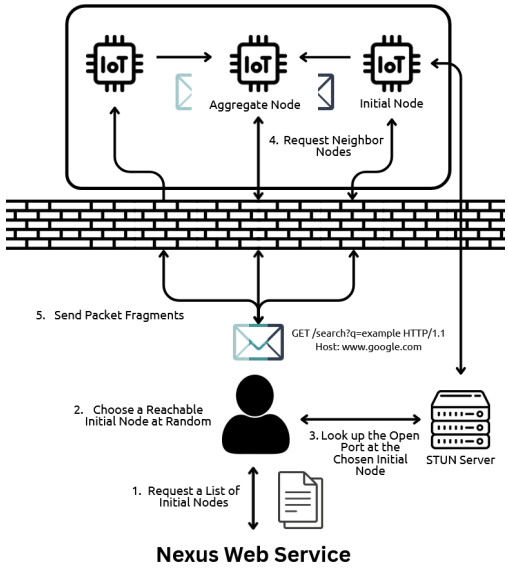


Figure 1: Communication between Client and Nodes

5 Nexus

In this paper, we show the implementation of our *Nexus Fragmentation Algorithm* alongside a theoretical *Nexus Network* distributed network, which is proposed to transmit and reassemble those fragments. We aim to meaningfully hamper real-time DPI and fingerprinting capabilities with this design, even if the encryption is only applied to the metadata.

We show the general workflow of *Nexus Network* in Figure 1. In this theoretical design, the client will fragment the original packet into an arbitrary number of pieces with our fragmentation algorithm and send them to different volunteer IoT devices. The volunteer nodes will decrypt the metadata in each fragment and discover the peer nodes that received the other corresponding pieces. The nodes will then elect an aggregate node amongst themselves through a pseudo-Paxos consensus algorithm [22], and send the reassembled packet to the destination. The same fragmentation procedure will occur from the aggregate node, distributed with the aforementioned nodes, and sends the fragments to the client's device for reassembly of the response data.

5.1 Types of Volunteer Node Devices

As *Nexus* is designed to maximize the effectiveness of a distributed circumvention approach, we make node-side *Nexus* deployable on a wide range of devices. This includes low-cost IoT devices (eg., ESP8266 and ESP32 SoCs, and Raspberry Pi single-board computers) often seen in Smart City equipment, such as smart trash cans, traffic lights, sensors, etc., and consumer-grade Smart Home devices such as Amazon Echo and Fire TV. In addition, we also consider

open-source projects such as *ESPHome* [13], which allows seamless integration of customized apps to be installed on devices running on ESP8266 or ESP32.

We categorize means of deployment into three levels, ranked by ease of deployment, from easiest to hardest:

(1) Side-loading *Nexus* on Commercial Devices
(2) Integrating Through Existing Platforms
(3) *Nexus* as Firmware

Side-loading Nexus on Commercial Devices The most intuitive place to deploy *Nexus* for the average user would be the Smart Home IoT devices they have purchased. However, this implies that volunteers expect the devices to function as intended while running *Nexus* in the background. Therefore, side-loading *Nexus*—acquiring it without going through the respective official application distributor—would be the simplest means of installation.

While not universally guaranteed to be compatible, we aim to ensure that *Nexus* reaches the average group of IoT consumers, who occupy the majority of the market [2]. However, this approach comes with a tradeoff of future uncertainty as Android, the core system many consumer IoT devices are based on, plans to close off venues to side-loading unofficial applications into their systems [24], albeit this has not been fully implemented at the time of this publication.

Integrating Through Existing Platforms When the cost of consumer-grade IoT devices goes beyond the budget, the next efficient solution would be to make use of existing platforms or systems designed for specific SoCs that allow modules to be installed. For instance, the aforementioned *ESPHome* allows *Nexus* to be installed as “Components", dedicated to a specific functionality in parallel to others. With platforms such as this, *Nexus* can be installed on low-power IoT devices with relative ease and high compatibility.

Nexus as Firmware For experienced volunteers, they have the choice to flash the entire device with the corresponding *Nexus* designated for a specific series of SoCs or architecture. Although low-cost, it requires volunteers to possess a certain level of expertise in embedded programming. In general, while not inherently difficult, this will nullify the original purpose of the device, whether it be a sensor with real-time data transmission or a surveillance camera installed in front of the door. Therefore, this means of installation is the most complicated and inconvenient for the average user.

5.2 Nexus Packet Fragmentation Algorithm

We propose that Nexus mainly utilizes the fragmentation algorithm based on a uniform-random bit length of *bit_n* for each fragment to define our delimiters with Algorithm 1, as shown in Figure 2. With this algorithm, we dissect the packets so that they are the bit size of *bit_n* % 8 != 0. This is proposed so that the censors cannot reassemble the packets without preemptively knowing the order in the case where they intercepted all fragments.

Furthermore, even with the naive example fragmentation algorithm 2 as a fallback, which is a keyword-based delineator, we show that Nexus still obstructs adversaries from fingerprinting traffic flow, since both produce different degrees of unpredictable packet lengths and stream directions, with semi-randomized fragmentation and multiple nodes for routing. We will demonstrate this in Section 7.1, and Section 8 further discusses the matter of formally verifying such hardness using *Indistinguishability Proof* [26].

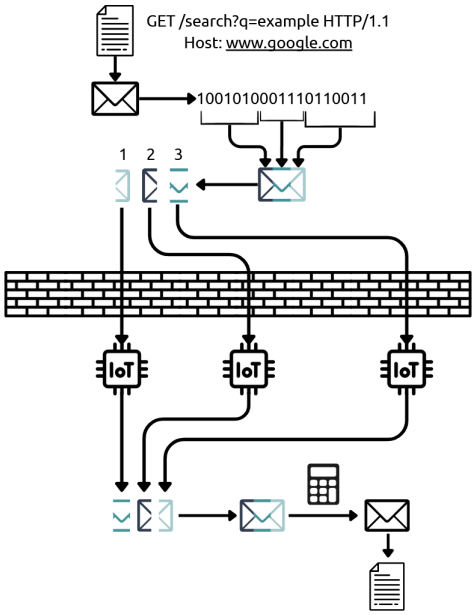


Figure 2: General Workflow of Fragmentation

Algorithm 1: FragmentationByUniformRandomLength

Input: Packet: byte sequence, MinLen: 1 bit, MaxLen: bit length of Packet - 1
Output: List of fragments after splitting the Packet

```
Fragments ← empty list;
randomLen ← uniformRandom(MinLen, MaxLen);
while Packet is not empty && randomLen ≤ bitLen(Packet)
 && 8 bits ≤ bitLen(Packet) do
    if randomLen % 8 == 0 then
        randomLen ← uniformRandom(MinLen, MaxLen);
        continue;
    fragment ← first randomLen bits of Packet;
    Append fragment to Fragments;
    Remove the first randomLen bits from Packet;
    randomLen ← uniformRandom(MinLen, MaxLen);
if bitLen(Packet) < 8 then
    fragment ← Packet;
    Append fragment to Fragments;
    Clear Packet;
return Fragments;
```

5.3 Metadata Header

During the handshake, the nodes respond to the client with an RSA public key, with which the client shares the AES-256 key and

Algorithm 2: FragmentationByKeywords

Input: Packet: byte sequence, Keywords: list of byte sequences
Output: List of fragments after splitting the Packet

```
foreach keyword in Keywords do
    index ← position of keyword in Packet;
    if index is found then
        chosenKeyword ← keyword;
        break;
if no keyword found then
    return {Packet} ;    // Return packet as a single fragment
else
    pivot ← index + ⌊ length(chosenKeyword) / 2 ⌋ ;
    LeftFragment ← substring of Packet from start to pivot;
    RightFragment ← substring of Packet from pivot to end;
    return FragmentationByKeywords(LeftFragment, Keywords) concatenated with FragmentationByKeywords(RightFragment, Keywords);
```

the initialization vector used to encrypt the protocol metadata, constructed as below:

$$metadata_{i,j} = \text{bytes}\big(sessionID + totalNum_{i,j} + orderIndex_{i,j} + padNum_{i,j} + nodeList_i\big) \tag{1}$$

$$|metadata_{i,j}| = L_{metadata} \geq 11 \text{ bytes}$$
$$sessionID = i \in \{P_i\}$$
$$totalNum_{i,j} = |\{PF_{i,j}\}|$$
$$orderIndex_{i,j} = j \in \{PF_{i,j}\}$$
$$nodeList_i = \{N\}$$

We denote the original unfragmented packets as P and fragmented packets of each P_i as $\{PF_{i,j}\}$. $L_{metadata}$ is the byte length of the metadata header of $PF_{i,j}$, the j-th fragment of the packet P_i. Subsequently, the *sessionID* is the unique ID for the packet P_i, $padNum_{i,j}$ is the byte length of the padding in fragment $PF_{i,j}$, and $nodeList_{i,j}$ contains the list of nodes N assigned to P_i which receive and relay fragments $\{PF_{i,j}\}$.

With the design, we allow the routing nodes to acquire the necessary information to coordinate with each other and reassemble the packets while maintaining secrecy against on-path adversaries.

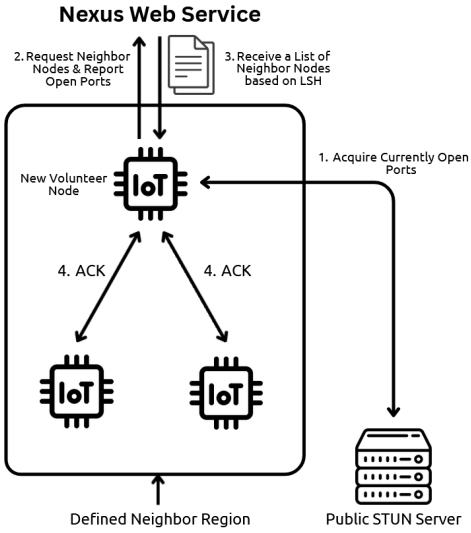


Figure 3: Bootstrapping Process of a New Volunteer Node

6 Theoretical Nexus Network

As part of *Nexus*' design, we also propose the theoretical *Nexus Network*, designed to both distribute relay node lists and route networks. We outline it here as the intended architecture that the fragmentation algorithm is designed to support. We hypothesize that this would impede censors' ability to detect circumvention traffic and discover the ordering of corresponding packets, as shown in our results in Section 7.1.

6.1 Node Coordination

As a crucial part of the infrastructure, volunteer nodes must have the means to communicate with each other and share information regarding ongoing packet sessions. To do this, we propose implementing a distribution model to share a list composed only of proximate nodes to which the user and volunteers can connect. The proximity can be defined by geographic distance, Euclidean distance based on encoding, or any arbitrary custom rules. This will allow us to minimize the implementation of a centralized connection list distribution system and attempt to minimize the risk of an impersonator (i.e., censors pretending to be a legitimate volunteer) acquiring the entire topology of the *Nexus Network*.

In addition, *Nexus Network* looks to adapt *Lox*, a distribution reputation system which "makes bridges available to untrusted users while leveraging users' trust networks."[31] With *Lox*, as nodes would have their reputation decreased the more their neighboring nodes get blocked, we can further hinder the effectiveness of impersonator nodes.

Volunteer Node Bootstrapping In the design of *Snowflake* by the *Tor Project*, it allows less tech-savvy volunteers to participate via Network Address Translation (NAT) Hole-Punching [6], a technique that enables outside connections to go through the volunteers' node devices behind routers' NAT by initiating an outbound connection to the entry nodes with open ports first. While it lowers

the technical barrier for the volunteers, the discrepancy in port-openness (whether the volunteer has their router port forwarded) implies that such a system requires “broker" servers to administer port-forwarded entry or exit nodes to send outbound connections to, decreasing the distributability.

For *Nexus Network*, we propose that all volunteer nodes serve as “brokers." Shown in Figure 3, for a volunteer outside the censored region, the selected volunteer’s device performs NAT Hole-punching by first connecting to a predetermined list of Public STUN servers and acquiring its currently open port to the public. After which, the volunteer will bootstrap by contacting the *Nexus*’ distribution server, denoted as the Nexus Web Service (NWS), and acquire the IP and port information along with a request for a non-exclusive subset(i.e., different subsets may contain same nodes) of IP addresses and open ports of neighbor nodes based on the distance defined by Locality-Sensitive Hashing (LSH) algorithms [20]. After receiving such a list, the volunteer attempts to establish handshakes using *Let’s Encrypt*, a Certificate Authority, with the neighboring nodes in the subset. Upon success, the node floods the neighboring nodes with the same subset and collectively forms a “neighborhood region". By doing so, even if a node previously connected to the network loses connection to the NWS, it would still be able to receive the updated peer node list through its current peers. Additionally, as each node will only have limited knowledge of the full *Nexus Network* topology, and can inform each other without necessarily going through the distribution server, no node constitutes a choke point of failure.

Client-Node Communication & File Reassembly As illustrated in Figure 1, when a client user installs *Nexus*, a list of potential initial nodes will be provided by the NWS for initiating handshakes, which will be updated in the case where the number of unreachable nodes exceeds a certain percentage threshold on the list. The user will then search for the IP address and look up the open port in sequence on the predetermined STUN servers, halting at the first reachable initial node. After the initial connection, the user will receive the subset list of neighbor nodes using a lossy channel (such as a fragmented channel or DNS) as rendezvous, per the design of Cawthon et al [7]. With the list, the user initiates a connection to all nodes. Upon success, the client starts fragmenting the original packets and send them to the nodes in non-deterministic order. The nodes must return an acknowledgment for every fragment received.

However, in case of permanent loss of connection to one or more but not all nodes, the client may request a new list from living nodes for a different subset list to continue transmission; in case of permanent loss of connection to all nodes, the bootstrapping process restarts from the beginning.

As demonstrated in Figure 4, with the aforementioned structure of encrypted metadata, shown in Equation 1, the nodes will decide the aggregate node based on file size or an arbitrarily defined process. The aggregate node will then accumulate all fragments, reassemble the original packet, and then be responsible for the downstream and upstream entry point between the final destination server and the client user. This stream of connection will persist over the course of the user’s continuous connection until actively terminated. However, the process will not complete unless

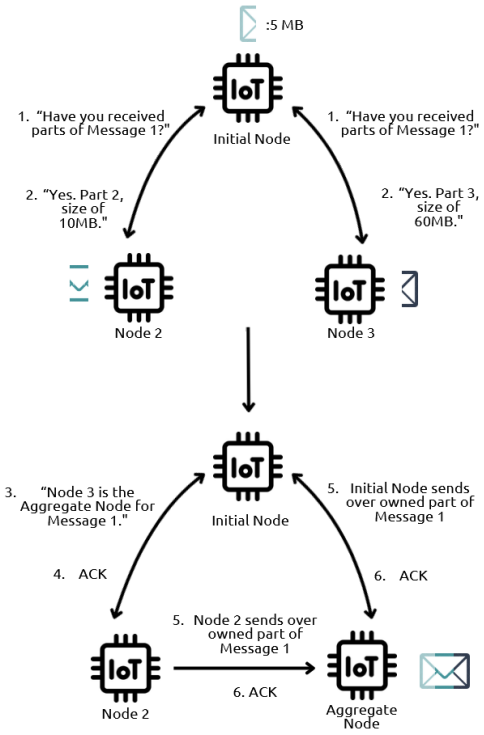


Figure 4: Inter-node Communication & Aggregate Node Election

all fragments are received, the number of which is indicated in the metadata.

In case of permanent loss to one or more nodes before receiving all fragments, the aggregate node will first request a new subset of neighbor nodes from currently alive nodes, then send an error message to the client to request a resend; the aggregate node will request a new list from the NWS instead should all other neighbor nodes permanently disconnect.

7 Evaluation

We design our testing environment as close to a real-world setting as possible. We deployed the clients on Alibaba and Tencent Lighthouse cloud servers located in China, simulating users within the censored regions, where traffic to prohibited websites is blocked when no circumvention was applied.

To account for computational constraints, only the informational metadata header of each fragment is encrypted. In addition, we utilized Algorithm 1—Fragmentation by Bit-stripping—during the evaluation to verify fingerprinting resistance.

For the test procedure, we make 10000 continuous requests via the *Nexus* in China using the test list by the Citizen Lab [21] within 24 hours. The purposes of the tests are to:

(1) Verify the feasibility of *Nexus Packet Fragmentation Algorithm*, thus setting a benchmark for the full implementation of *Nexus*.

Table 1: 1-node (FireTV Stick)

Attributes	Values
Average Packet Length	651.31 Bytes
Median Packet Length	66 Bytes
Packet Length Coefficient of Variation	2.1815
Payload Bit Entropy [36]	3.9825 Bits
Stream Direction Shannon Entropy	0.7908 Bits/1 Bit (Binary)
Average Outbound Burst	1.14 Packets/Burst
Average Inbound Burst	1.65 Packets/Burst
Average Webpage Load Time	2.8241 Seconds

Table 2: 1-node (Raspberry Pi Zero 2 W)

Attributes	Values
Average Packet Length	317.58 Bytes
Median Packet Length	312 Bytes
Packet Length Coefficient of Variation	4.9408
Payload Bit Entropy [36]	3.9396 Bits
Discrete Stream Direction Shannon Entropy	0.9603 Bits/1 Bit (Binary)
Average Outbound Burst	2.36 Packets/Burst
Average Inbound Burst	1.51 Packets/Burst
Average Webpage Load Time	6.7297 Seconds

(2) Test GFW's ability to detect any fingerprint and categorize it as an anomaly.

Furthermore, we deployed two modes of testing: one-node mode and two-node mode.

For tests with only one node, accounting for the worst-case scenario where there is only one alive in the "neighborhood" network, we utilize a FireTV Stick and a Raspberry Pi Zero 2 W to simulate possible volunteer nodes with two different configurations, with low cost and computational power, located in the United States. This was designed to verify the effectiveness of the fragmentation algorithm alone in censorship circumvention, and collect differences in fingerprint features depending on the deployment devices.

For two-node tests, we deployed *Nexus* on both a Raspberry Pi Zero 2 W and a System76 Meer6, also located in the United States, to test the cross-compatibility with more capable computing devices, and verify its effect in multi-path routing. This setup follows the snapshot state where the aggregate node is elected, and the users are sending fragments to both the relay nodes and aggregate nodes, where the aggregate nodes will aggregate all fragments and send the response back to the user.

These setups allow us to establish a baseline for *Nexus*' general usability when deployed in a real home environment and on low-power devices, and its ability to disrupt known fingerprinting techniques.

7.1 Results

Fingerprinting & Bypassing Circumvention As shown in Table 1, Table 2, and Table 3, with the fragmentation algorithm, we have achieved a high coefficient of variation (CV) in packet length and network stream direction Shannon entropy (client to node, or vice versa) to account for commonly deployed fingerprinting techniques [8, 23, 25, 30]. Furthermore, noticeably, attributes vary

Table 3: 2-Node (Raspberry Zero 2 W, System76 Meer6)

Attributes	Values
Average Packet Length	319.62 Bytes
Median Packet Length	314 Bytes
Packet Length Coefficient of Variation	4.9434
Payload Bit Entropy [36]	3.9384 Bits
Discrete Stream Direction Shannon Entropy	0.9669 Bits/1 Bit (Binary)
Average Outbound Burst	2.46 Packets/Burst
Average Inbound Burst	1.59 Packets/Burst
Average Webpage Load Time	7.8894 Seconds

drastically across device types (ie., FireTV vs. Raspberry), emphasizing the importance of cross-compatibility in further dampening the fingerprints of *Nexus*. For two-node mode, the multi-node design also shows a high Shannon entropy of the directions of these fragment packets. This shows that a multi-path routing scheme also impedes on-path fingerprinting, even assuming that the censors can identify related packets. However, surprisingly, despite the fact that the Payload Bit Entropy falls within the blocking heuristics discovered by Wu et al. [36], we found no sign of blocking and all requests were successful.

In addition, as examples of how real-life fingerprinting can be deployed facing large traffic, we curated our PCAP files from the testing into CSV flow files using CICFlowMeter [16], then incorporated two commonly used unsupervised fingerprinting techniques: K-means clustering and Isolation Forest. We wish to ask two questions with these two methods: 1. If the censors observe *Nexus* traffic in the mix of normal traffic, can they classify it with common fingerprinting features? 2. If the censors have not seen *Nexus*, can we detect it with common fingerprinting features?

Shown in Appendix A, where the red crosses represent each *Nexus* traffic flow not classified in a cluster consisting only of *Nexus* traffic, neither method meaningfully distinguishes *Nexus*' traffic from the baseline traffic (random requests to websites over TLS and idle noises). K-means clustering could only classify around 1% of all *Nexus* traffic in purely *Nexus* clusters at k=16, and a binary (k=2) classifier did not classify any purely *Nexus* cluster. Isolation Forest's detection rate for either test mode is at most 3.84%, demonstrating a low success rate in detecting *Nexus*' traffic as an anomaly, given normal background traffic. However, as a trade-off, we also observed an increase in average webpage load time in two-node mode. This could be caused by a combination of programming language limitations, network instability (as the nodes are deployed in home network environments), and more routing paths.

In conclusion, we found that the algorithm and multi-node routing enabled successful circumvention of the GFW without the common behaviors indicating blocking (e.g., RST response, packet drop, DNS pollution, etc.), and the testbed client accessed prohibited websites and resources despite minimal encryption. In addition, we also found no significant degradation in system performance on any of the testbed devices other than slower network transmission caused by occupied bandwidth during testing.

8 Discussions

Despite initial successes, we recognize the following limitations in the design of the current prototype and test environments, elucidating potential areas for improvement in the future.

Test Environments for Nexus' Distributed System Due to the crowd-sourced nature of the design and to keep within the scope of the project, we did not test the node-to-node distribution function on our current prototype. For future research, we aim to further explore the flood-based distribution mechanism and look forward to community feedback on real-life implementations.

Impersonator Nodes Although *Nexus* mitigates the threat of impersonator nodes, it was not eliminated completely. While *Tor* protects the clients against malicious volunteer nodes with *Entry Guards*, it decreases the clients' chance of being profiled but does not conceal the fact that the clients attempted to connect to the *Tor* network; it also does not prevent intermediary nodes from being discovered and consequently blocked.

Similarly, *Nexus* passively protects the client through the combination of fragmentation and the node-to-node distribution mechanism, ensuring that no single node will acquire all packet fragments or the entire topology of the *Nexus* network. However, this is still a play of probability in the hope that the clients will not connect to the malicious nodes with no guarantee, particularly when *Nexus* is designed with a self-managed network in mind. Even with deploying the *Lox* reputation system [31], given the vast resources state actors possess, there is still possibility that they simply overrun the system by sheer number, at which point the issue would be their willingness to do so.

Indistinguishability Proof While the test results of *Nexus*' fragmentation algorithm achieved a promising start, we do not yet have the proper verification formula or toolkit to formally verify its mathematical complexity. This exposes a long-lasting vulnerability in censorship research at the theoretical level, requiring recurrent trial-and-error processes along the way. Building upon this, future research may focus on general-use methodologies and procedures to formulate proof of undetectability and indistinguishability for newly developed circumvention models. For instance, the *VeriWeird* project provided a novel approach coined as *simulatable indistinguishability proofs* (SIPs), which formalizes covert communication channels based on the encoding of their properties in the "simulation-based cryptographic proof paradigm." [26] Such a formalization would not only accelerate future censorship research projects without compromising reliability, but would also help technical communities to rigorously verify the feasibility of a novel approach.

9 Ethics

9.1 Volunteer & User Privacy

This project respects the privacy of all potential beneficiaries. Therefore, we did not solicit third-party participation in our testing process. We also inform potential interested developers about possible breaches of personal information (such as IP addresses) from both the clients and the volunteers, as we have yet to discover a method to guarantee the prevention of potentially malicious nodes. This issue is much more pronounced when the effectiveness in disrupting traffic flow fingerprinting of *Nexus* could be improved, should there be intermediary nodes within the censored region, exposing volunteers of these nodes to potential repercussions.

Furthermore, without a robust validation system to prevent censors from posing as legitimate volunteers, the risk of exposing client users' information persists as it does for *Tor* [27]. As such, there exists a possibility that a user may happen to connect to an impersonator Initial Node, which can also be selected as an Aggregate Node. Although low, the possibility alone can lead client users to legal issues, accusations, or even incarceration within censored regions [15]. The severity of the risk compels the ethical obligation to deter the public from implementing the model of the paper without any alteration, and to be wary of possible consequences.

However, as mentioned in Section 8, one path to alleviate the concern is to implement a trust-ranking system based on features that do not fully break the anonymity of volunteers (e.g., IP address reputation, length of valid volunteering time, etc.). Such a system would discourage censors from devoting resources to deploy malicious nodes by rendering the method inefficient, whilst not drastically degrading the principles of the distributed network by giving complete administration over the Initial Nodes and Aggregate Nodes to a single entity (i.e., *Nexus*' official team). At this time, the proposed *Nexus* model reduces risks for volunteers with a node assignment distribution mechanism, where no one volunteer node will gain a list of nodes for the entire topology given a specific session, therefore limiting the effectiveness of impersonator nodes. Nevertheless, it is still inadvisable to deploy *Nexus* as a participant within the censored region.

9.2 Use of IP addresses from Public Spaces

We acknowledge that *Nexus* might be deployed in public spaces accompanied by hole-punching nodes, as IoT devices are often covert, and a public network could reduce the possible risk of personal identity breaches. The authors discourage such actions due to the aforementioned risk involved, as "innocent bystanders"—i.e., the owners of such public places— will face possible legal liability unwillingly and unknowingly. Therefore, we ask that interested parties refrain from acting in this capacity unless supported by the proprietor.

10 Conclusions

In this paper, we present the *Nexus Packet Fragmentation Algorithm* and its multi-path routing as the first validated components of *Nexus*, a proposed distributed IoT-based circumvention system. We evaluated the algorithm and showed initial success in both circumvention and limited fingerprinting, building a promising foundation. With the prototype, we propose a new distributed approach for future censorship research, eventually moving away from mimicry-based obfuscation, instead towards increasing computational burdens for on-path censors in both packet inspection and real-time traffic surveillance.

AI Use

The authors have utilized Claude to assist in the creation of Python files for testing and statistics collection.

References

[1] Talha Barkaat Ahmad. 2024. Laptop Prices Dataset - October 2023. https://www.kaggle.com/datasets/talhabarkaatahmad/laptop-prices-dataset-october-2023

[2] Tanweer Alam. 2018. A Reliable Communication Framework and Its Use in Internet of Things (IoT). 3 (05 2018).

[3] Anonymous. 2014. Towards a Comprehensive Picture of the Great Firewall's DNS Censorship. In *IEEE Symposium on Foundations of Computational Intelligence*. https://api.semanticscholar.org/CorpusID:8688873

[4] Anonymous, Arian Akhavan Niaki, Nguyen Phong Hoang, Phillipa Gill, and Amir Houmansadr. 2020. Triplet Censors: Demystifying Great Firewall's DNS Censorship Behavior. In *10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20)*. USENIX Association. https://www.usenix.org/conference/foci20/presentation/anonymous

[5] Kevin Bock, Yair Fax, Kyle Reese, Jasraj Singh, and Dave Levin. 2020. Detecting and Evading Censorship-in-Depth: A Case Study of Iran's Protocol Whitelister. In *10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20)*. USENIX Association. https://www.usenix.org/conference/foci20/presentation/bock

[6] Cecylia Bocovich, Arlo Breault, David Fifield, Serene, and Xiaokang Wang. 2024. Snowflake, a censorship circumvention system using temporary WebRTC proxies. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 2635–2652. https://www.usenix.org/conference/usenixsecurity24/presentation/bocovich

[7] Eleanor Cawthon and David Fifield. 2026. Fountain codes in censorship circumvention rendezvous. *Free and Open Communications on the Internet*.

[8] Rajarshi Roy Chowdhury and Pg Emeroylariffion Abas. 2022. A survey on device fingerprinting approach for resource-constraint IoT devices: Comparative study and research challenges. *Internet of Things* 20 (2022), 100632. https://doi.org/10.1016/j.iot.2022.100632

[9] Fredrik Dahlqvist, Mark Patel, Alexander Rajko, and Jonathan Shulman. 2019. Growing opportunities in the Internet of Things. *McKinsey & Company* 22 (2019).

[10] Roger Dingledine, Nick Mathewson, and Paul Syverson. 2004. Tor: the second-generation onion router. In *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13* (San Diego, CA) *(SSYM'04)*. USENIX Association, USA, 21.

[11] Jason A Donenfeld. 2017. WireGuard: Next Generation Kernel Network Tunnel.. In *NDSS*. 1–12.

[12] Roya Ensafi, Philipp Winter, Abdullah Mueen, and Jedidiah R Crandall. 2014. Large-scale Spatiotemporal Characterization of Inconsistencies in the World's Largest Firewall. *arXiv preprint arXiv:1410.0735* (2014).

[13] ESPHome. 2018. ESPHome. https://github.com/esphome/esphome.

[14] Shencha Fan, Jackson Sippe, Sakamoto San, Jade Sheffey, David Fifield, Amir Houmansadr, Elson Wedwards, and Eric Wustrow. 2025. Bleeding Wall: A Hematologic Examination on the Great Firewall. *Network and Distributed System Security (NDSS)* (2025).

[15] Benjamin Haas. 2017. Man in China sentenced to five years' jail for running VPN. *The Guardian* (Dec 2017). https://www.theguardian.com/world/2017/dec/22/man-in-china-sentenced-to-five-years-jail-for-running-vpn

[16] Arash Habibi Lashkari. 2018. CICFlowmeter-V4.0 (formerly known as ISCXFlowMeter) is a network traffic Bi-flow generator and analyser for anomaly detection. https://github.com/ISCX/CICFlowMeter. https://doi.org/10.13140/RG.2.2.13827.20003

[17] Nguyen Phong Hoang, Arian Akhavan Niaki, Jakub Dalek, Jeffrey Knockel, Pellaeon Lin, Bill Marczak, Masashi Crete-Nishihata, Phillipa Gill, and Michalis Polychronakis. 2021. How Great is the Great Firewall? Measuring China's {DNS} Censorship. In *30th USENIX Security Symposium (USENIX Security 21)*. 3381–3398.

[18] Amir Houmansadr, Chad Brubaker, and Vitaly Shmatikov. 2013. The Parrot Is Dead: Observing Unobservable Network Communications. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy (SP '13)*. IEEE Computer Society, USA, 65–79. https://doi.org/10.1109/SP.2013.14

[19] Muhammad Iqbal and Imam Riadi. 2019. Analysis of security virtual private network (VPN) using openVPN. *International Journal of Cyber-Security and Digital Forensics* 8, 1 (2019), 58–65.

[20] Omid Jafari, Preeti Maurya, Parth Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. 2021. A survey on locality sensitive hashing algorithms and their applications. *arXiv preprint arXiv:2102.08942* (2021).

[21] Citizen Lab and Others. 2014. URL testing lists intended for discovering website censorship. https://github.com/citizenlab/test-lists https://github.com/citizenlab/test-lists.

[22] Leslie Lamport. 2001. Paxos Made Simple. *ACM SIGACT News (Distributed Computing Column) 32, 4 (Whole Number 121, December 2001)* (December 2001), 51–58. https://www.microsoft.com/en-us/research/publication/paxos-made-simple/

[23] Ruoyu Li, Qing Li, Tao Lin, Qingsong Zou, Dan Zhao, Yucheng Huang, Gareth Tyson, Guorui Xie, and Yong Jiang. 2024. DeviceRadar: Online IoT Device Fingerprinting in ISPs Using Programmable Switches. *IEEE/ACM Transactions on Networking* 32, 5 (2024), 3854–3869. https://doi.org/10.1109/TNET.2024.3398778

[24] Keep Android Open. 2026. Keep Android Open. https://keepandroidopen.org/.

[25] R. Perdisci, T. Papastergiou, O. Alrawi, and M. Antonakakis. 2020. IoTFinder: Efficient Large-Scale Identification of IoT Devices via Passive DNS Traffic Analysis. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. Genoa, Italy.

[26] Vitor Pereira, Ahmed Irfan, Vinod Yegneswaran, Nick Feamster, Prateek Mittal, and Vitaly Shmatikov. 2025. Position Paper: A Case for Machine-Checked Verification of Circumvention Systems. *Free and Open Communications on the Internet* (2025).

[27] Tor Project. 2006. *Tor Project* (2006). https://community.torproject.org/threat-model/threat-positioning/network/

[28] Ram Sundara Raman, Mona Wang, Jakub Dalek, Jonathan Mayer, and Roya Ensafi. 2022. Network measurement methods for locating and examining censorship devices. In *Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies* (Roma, Italy) *(CoNEXT '22)*. Association for Computing Machinery, New York, NY, USA, 18–34. https://doi.org/10.1145/3555050.3569133

[29] Espressif Systems. 2025. ESP SoCs. https://www.espressif.com/en/products/socs

[30] Jay Thom, Nathan Thom, Shamik Sengupta, and Emily Hand. 2022. Smart Recon: Network Traffic Fingerprinting for IoT Device Identification. In *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*. 0072–0079. https://doi.org/10.1109/CCWC54503.2022.9720739

[31] Lindsey Tulloch and Ian Goldberg. 2023. Lox: Protecting the social graph in bridge distribution. *Proc. Priv. Enhancing Technol.* 2023, 1 (Jan. 2023), 494–509.

[32] Vasilis Ververis, Tatiana Ermakova, Marios Isaakidis, Simone Basso, Benjamin Fabian, and Stefania Milan. 2021. Understanding Internet Censorship in Europe: The Case of Spain. In *Proceedings of the 13th ACM Web Science Conference 2021* (Virtual Event, United Kingdom) *(WebSci '21)*. Association for Computing Machinery, New York, NY, USA, 319–328. https://doi.org/10.1145/3447535.3462638

[33] Ryan Wails, George Arnold Sullivan, Micah Sherr, and Rob Jansen. 2024. On Precisely Detecting Censorship Circumvention in Real-World Networks. *Proceedings 2024 Network and Distributed System Security Symposium* (2024). https://api.semanticscholar.org/CorpusID:266696753

[34] Zachary Weinberg, Diogo Barradas, and Nicolas Christin. 2021. Chinese Wall or Swiss Cheese? Keyword filtering in the Great Firewall of China. In *Proceedings of the Web Conference 2021* (Ljubljana, Slovenia) *(WWW '21)*. Association for Computing Machinery, New York, NY, USA, 472–483. https://doi.org/10.1145/3442381.3450076

[35] WireGuard. 2025. Known Limitations - WireGuard. https://www.wireguard.com/known-limitations/

[36] Mingshi Wu, Jackson Sippe, Danesh Sivakumar, Jack Burg, Peter Anderson, Xiaokang Wang, Kevin Bock, Amir Houmansadr, Dave Levin, and Eric Wustrow. 2023. How the Great Firewall of China Detects and Blocks Fully Encrypted Traffic. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 2653–2670. https://www.usenix.org/conference/usenixsecurity23/presentation/wu-mingshi

[37] Mingshi Wu, Ali Zohaib, Zakir Durumeric, Amir Houmansadr, and Eric Wustrow. 2025. A Wall Behind A Wall: Emerging Regional Censorship in China . In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1363–1380. https://doi.org/10.1109/SP61157.2025.00152

[38] Project X. 2026. VLESS (XTLS Vision Seed). https://xtls.github.io/en/config/outbounds/vless.html.

[39] Diwen Xue, Michalis Kallitsis, Amir Houmansadr, and Roya Ensafi. 2024. Fingerprinting Obfuscated Proxy Traffic with Encapsulated TLS Handshakes. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA.

[40] Diwen Xue, Benjamin Mixon-Baca, ValdikSS, Anna Ablove, Beau Kujath, Jedidiah R. Crandall, and Roya Ensafi. 2022. TSPU: Russia's Decentralized Censorship System. In *Proceedings of the 22nd ACM Internet Measurement Conference*. Association for Computing Machinery, Nice, France.

[41] Diwen Xue, Reethika Ramesh, Arham Jain, Michalis Kallitsis, J. Alex Halderman, Jedidiah R. Crandall, and Roya Ensafi. 2022. OpenVPN Is Open to VPN Fingerprinting. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association.

A Nexus Test Result Graphs

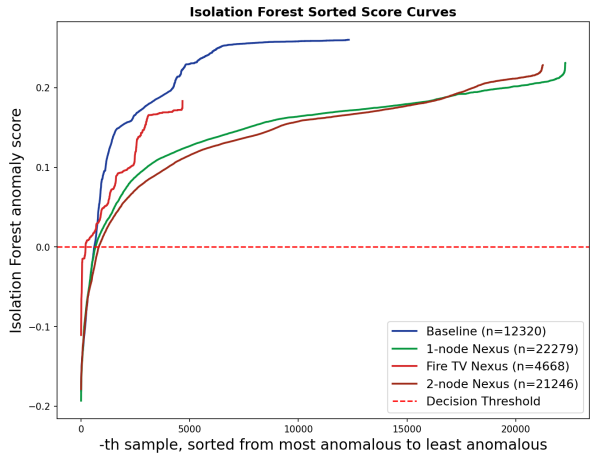


Figure 5: Anomaly Isolation Forest (1-node v.s 2-node v.s Baseline)

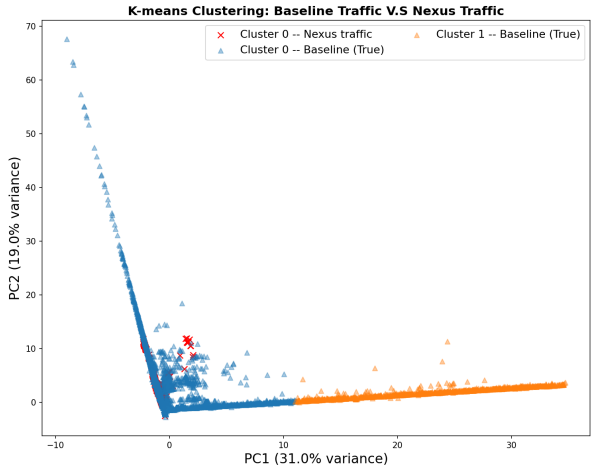


Figure 6: K-means (FireTV Stick v.s Baseline, k=2)

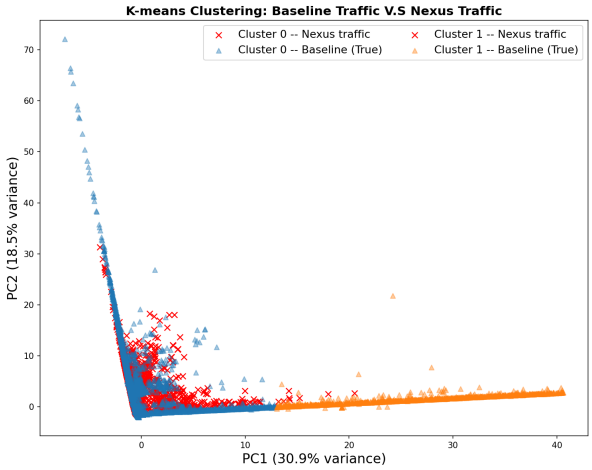


Figure 7: K-means (Raspberry Pi v.s Baseline, k=2)

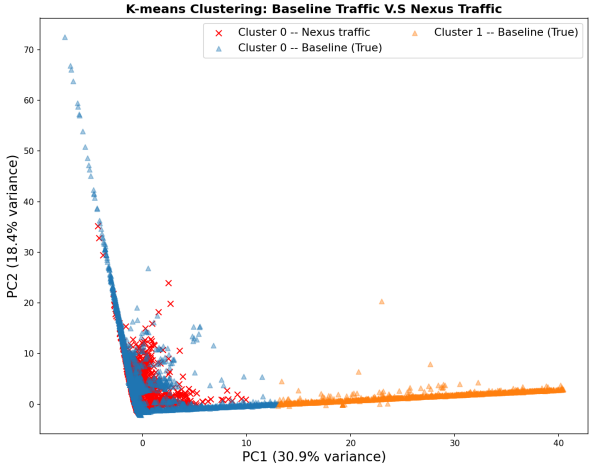


Figure 8: K-mean (Raspberry Pi + System76 v.s Baseline, k=2)

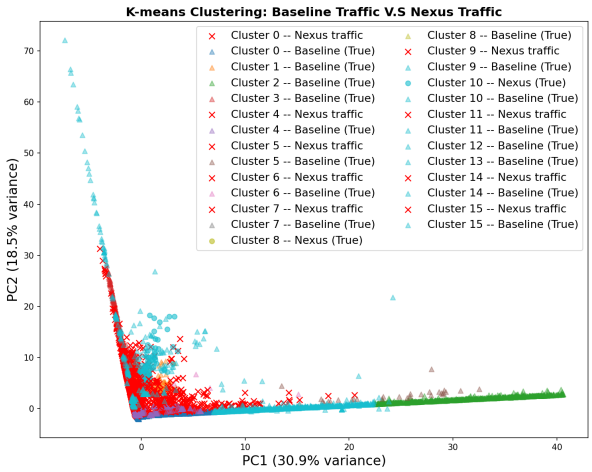


Figure 9: K-means (Raspberry Pi v.s Baseline, k=16)

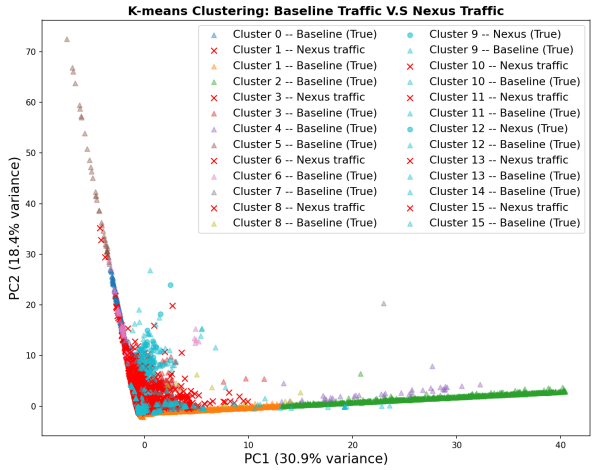


Figure 10: K-means (Raspberry Pi + System76 v.s Baseline, k=16)

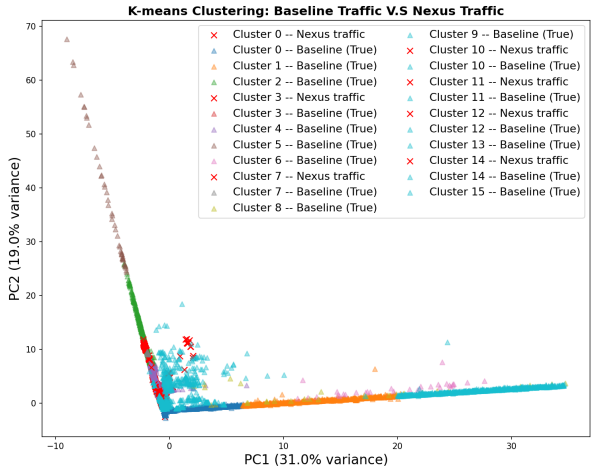


Figure 11: K-means (FireTV Stick v.s Baseline, k=16)